

— Linux essentials

*Containers (Learning odern Linux,
Hausenblas M.)*



— Introduction

— Wat zijn containers?

Een **container** is een groep Linux-processen die geïsoleerd draait dankzij *namespaces*, *cgroups* en eventueel *CoW-bestandssystemen*. Containers zorgen ervoor dat applicaties werken met hun eigen afhankelijkheden, los van het basissysteem.

Typische toepassingen:

- Lokale testen en ontwikkeling
- Microservices in Kubernetes

Als eindgebruiker werk je meestal via hogere tools zoals **moderne pakketbeheerders (apt, dnf etc.)**.

— Historiek van containers in Linux

Containers bestaan al lang, maar werden pas populair vanaf 2014 met **Docker**.

Eerdere implementaties:

- Linux-VServer (2001)
- OpenVZ (2005)
- LXC (2008)
- Imctfy (2013)

Al deze systemen maken gebruik van bestaande kernelmechanismen zoals namespaces en cgroups.

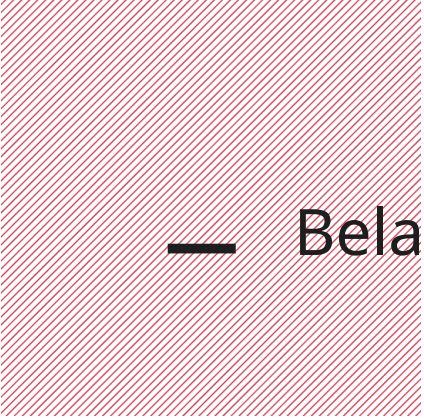
— Wat bracht Docker?

Docker zorgde voor de doorbraak door:

- Een standaardformaat voor container images te introduceren
- Een gebruiksvriendelijke CLI te bieden (zoals docker run)

Hieruit ontstond de Open Container Initiative (OCI), met 3 standaarden:

- Runtime-spec: hoe containers draaien en stoppen
- Image-spec: opbouw van images met lagen en metadata
- Distribution-spec: hoe images gedeeld worden via registries



— Belangrijk kenmerk: Immutability

Containers zijn **immutabel**: je wijzigt een container niet tijdens gebruik. Elke wijziging vereist een nieuwe image.

— Namespaces en Cgroups

— Namespaces – Resource-oriëntatie

Linux startte met een **globale kijk op resources** (PID's, filesystems, netwerken...). Namespaces zorgen ervoor dat een proces slechts een **lokale, geïsoleerde weergave** van bepaalde resources heeft.

Let op: isolatie betekent **wat een proces kan zien**, niet noodzakelijk wat het kan beïnvloeden (geen volledige sandboxing).

— Namespace-syscalls

Namespaces worden aangemaakt of beheerd via drie kernel-syscalls:

- `clone(flags)`: maak nieuw proces met aparte namespace(s)
- `unshare(flags)`: splits namespace van bestaand proces af
- `setns(fd, nstype)`: laat bestaand proces toetreden tot een namespace

Elke syscall gebruikt `CLONE_-`flags om het type namespace te bepalen.

— Overzicht van CLONE_-flags

Flag	Doel	Kernelversie
CLONE_NEWNS	Eigen mount points	2.4.19
CLONE_NEWUTS	Eigen hostname/domain	2.6.19
CLONE_NEWIPC	Eigen IPC (System V, POSIX MQ)	2.6.19
CLONE_NEWPID	Eigen PID-ruimte	2.6.24
CLONE_NEWNET	Eigen netwerkstack	2.6.29
CLONE_NEWUSER	UID/GID mapping	3.8
CLONE_NEWCGROUP	Eigen cgroup-hiërarchie	4.6

— Namespaces inspecteren

Namespaces zijn zichtbaar in `/proc/$PID/` of via tools zoals:

- `lsns` # Overzicht van actieve namespaces per proces
- `cat /proc/$$/ns/` # Symbolische links naar namespace-structuren

Elke link toont een inode-ID waarmee je kan zien welke processen dezelfde namespace delen.

— Wat zijn cgroups?

Control groups (cgroups) zijn een kernelmechanisme om processen hiërarchisch te groeperen en hun **resourcegebruik te beheren en op te volgen**.

Ze worden gebruikt om:

- CPU-, geheugen- en I/O-gebruik te **beperken**
- Het aantal processen of toegang tot devices te **beheersen**
- Resourcegebruik van groepen processen **te monitoren**

Denk aan cgroups als de declaratieve laag; de kernel voert limieten af via **controllers**.

— cgroups v1 – Opbouw en beperkingen

Versie 1 werd incrementeel uitgebouwd zonder uniform ontwerp. Elke controller had zijn eigen hiërarchie.

Voorbeelden van v1-controllers:

- cpu: CPU-verdeling (CFS bandwidth)
- cpuset: toewijzen van CPU's/geheugen
- memory: RAM-gebruik isoleren
- devices: toegang tot device-files beheren
- freezer: batch-jobs tijdelijk bevriezen
- net_cls, net_prio: netwerkverkeer labelen of prioriteren
- pids: max. aantal processen instellen

Elke controller had aparte mappen in `/sys/fs/cgroup/`.

— cgroups v2 – Moderne aanpak

Versie 2 is een **herontwerp** met één hiërarchie en uniforme configuratie:

- Eén centrale boomstructuur
- Eenduidige interface voor alle controllers
- Beter gedocumenteerd en minder foutgevoelig
- Wordt standaard gebruikt in o.a. Fedora, Arch, Ubuntu 21.10+

Belangrijkste controllers in v2:

- cpu: verdeelt CPU-tijd (gewicht of limiet)
- memory: limieten op RAM, caches, TCP-buffers
- io: I/O-perf (bytes/sec of IOPS)
- pids: proceslimiet
- cpuset, hugetlb, rdma, devices: zoals v1, maar eenduidiger

— Werken met cgroups in de praktijk

Inspectie via systemd-tools:

- `systemctl status` toont de cgroup-boom per service
- `systemd-cgtop` geeft live inzicht in CPU/RAM-verbruik per cgroup

Voorbeeld:

```
systemd-run --unit=beperkt --property=MemoryMax=100M bash
```

→ start een bash-shell in een nieuwe cgroup met 100MB RAM-limiet

De bestanden en limieten staan in `/sys/fs/cgroup/` (v2 heeft één directoryboom).

— Copy-on-Write (CoW) Filesystems

Een **Copy-on-Write filesystem** wordt gebruikt tijdens het **bouwen van container images**.

Het idee:

- Bestanden worden pas gekopieerd als ze **gewijzigd** worden
- Ongewijzigde bestanden blijven **gedeeld tussen lagen**
- Dit maakt images **klein, efficiënt en snel te bouwen**

Toepassing:

- Container images bestaan uit meerdere lagen
- Elke laag is een CoW-laag bovenop de vorige
- Meestal gecombineerd met **bind mounts** om componenten dynamisch samen te voegen

Gevolg: je krijgt één **zelfstandig bestand** dat alle code en afhankelijkheden bevat — klaar om te distribueren.

— Docker

— Wat is Docker?

Docker (gelanceerd in 2014) is een **gebruiksvriendelijke tool** om applicaties en hun afhankelijkheden te **verpakken en uitvoeren** in containers.

Wat maakt Docker bijzonder?

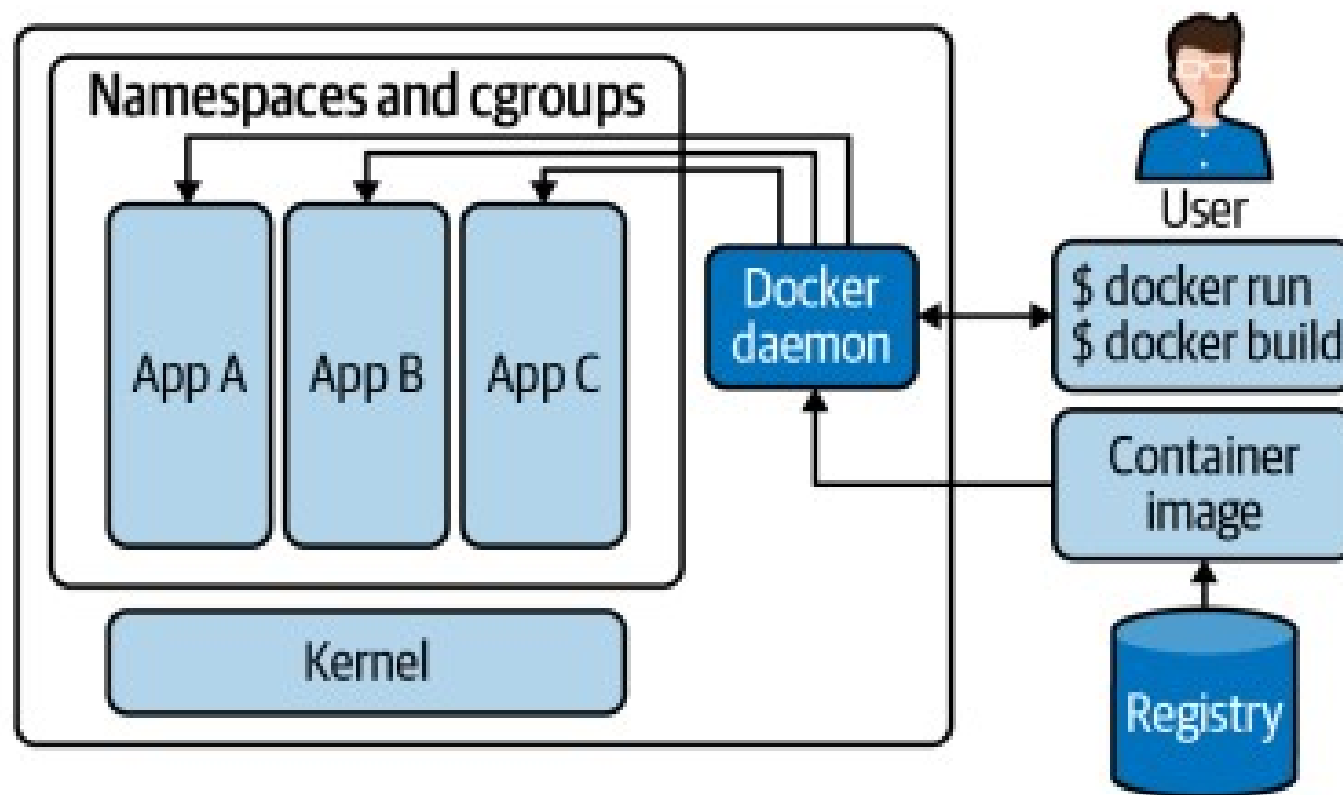
- Gebruikt bestaande Linux-functies: **namespaces, cgroups, CoW-filesystems, bind mounts**
- Verbergt de **complexiteit** van deze mechanismen achter eenvoudige commando's

Met Docker kun je snel containers starten, lokaal of in de cloud:

```
docker run ubuntu
```

Belangrijke concepten:

- **Image:** kant-en-klaar sjabloon met software en omgeving
- **Container:** actieve instantie van een image (runtime)



— Docker: image vs. container

Docker werkt met twee hoofdconcepten:

- **Image:** kant-en-klaar pakket met applicatie + afhankelijkheden
- **Container:** actieve instantie van die image (runtime)

Je bouwt een image → start er een container van → beheert die via de Docker CLI.

— Container image (build time)

Een **image** is een gecomprimeerd archief:

- Bestaat uit **JSON-metadata** en **bestandslagen**
- Wordt opgehaald uit een **registry** (bv. Docker Hub, AWS ECR)
- Is read-only en dient als sjabloon voor containers

Beheer:

- `docker build -t mijnapp:1.0 .`
- `docker pull ubuntu:22.04`
- `docker push mijnrepo/mijnapp:1.0`

— De container (runtime)

Een **container** is de actieve, gemonteerde versie van een image:

- Beheerbaar via CLI (docker run, stop, rm, ...)
- Draait via de Docker daemon (achtergrondproces)

Voorbeelden:

- `docker run -d --rm nginx:1.21` # start tijdelijke container
- `docker ps -a` # toon alle containers
- `docker inspect <id>` # detailinfo, zoals IP-adres

— Installatie

Installeer docker in windows

Installeer docker in Linux:

```
curl -fsSL https://get.docker.com | sh
```

curl -fsSL: downloadt het installatie-script
| sh: voert het script rechtstreeks uit in een shell

Om Docker zonder sudo te gebruiken:
sudo usermod -aG docker \$USER

Daarna moet je uit- en opnieuw inloggen om deze wijziging actief te maken.

— Docker CLI – veelgebruikte commando's

Commando	Doel
docker run	Start een container
docker ps	Toon actieve containers
docker build	Maak lokaal een image aan
docker pull	Download image uit registry
docker push	Upload image naar registry
docker images	Toon lokale images
docker image prune -a	Verwijder ongebruikte images

— Dockerfile – Wat is het?

Een **Dockerfile** is een tekstbestand met **instructies** om een container image laag per laag op te bouwen.

Belangrijkste bouwstenen:

- **FROM:** basisimage
- **LABEL:** metadata
- **ARG / ENV:** variabelen
- **COPY / RUN:** bestanden kopiëren, commando's uitvoeren (build time)
- **CMD / ENTRYPOINT:** wat het containerproces moet starten (runtime)

— Voorbeeld: Dockerfile met systeeminfo-script

Stap 1: Maak het script *systeeminfo.sh* aan

```
#!/bin/bash
```

```
echo "Systeeminformatie:"
```

```
uname -a
```

Zorg dat het script uitvoerbaar is:

```
chmod +x systeeminfo.sh
```

— Voorbeeld: Dockerfile met systeeminfo-script

```
FROM ubuntu:20.04
```

Gebruikt expliciet Ubuntu 20.04 als basis

```
LABEL
```

```
org.opencontainers.image.authors="JouwNaam"
```

Geeft het label *auteur* jouw naam

```
COPY systeeminfo.sh /app/
```

Kopieert een shellscript naar de container

```
WORKDIR /app
```

Verplaatst naar werkmap /app

```
RUN chown -R 1001:1 /app
```

Past rechten aan en draait niet als root

```
USER 1001
```

```
ENTRYPOINT ["/app/systeeminfo.sh"]
```

ENTRYPOINT zorgt dat het script uitvoert bij docker run

— Stap 3: Bouw de container image

Open PowerShell of CMD in de map en voer uit:

```
docker build -t systeeminfo:1 .
```

Controleer of de image aanwezig is:

```
docker images
```

— Stap 4: Voer de container uit

```
docker run systeeminfo:1
```

Je krijgt output zoals:

Systeeminformatie:

Linux 123abc 5.15.0-123-generic #124-Ubuntu SMP x86_64 GNU/Linux

— Foutmelding

Deze foutmelding:

```
exec /app/systeeminfo.sh: no such file or directory
```

komt vaak voor in Docker als:

- Het bestand **niet op de juiste plaats is gekopieerd**.
- Of als het **line-ending formaat fout is** (Windows vs. Linux).
- Of als de **bestandspaden/permissions** niet goed zijn ingesteld.

— Oplossing

Controleer line endings

Windows gebruikt standaard CRLF, maar Linux verwacht LF. Docker leest het bestand in Linux-context.

Oplossing: converteer naar Unix line endings (LF):

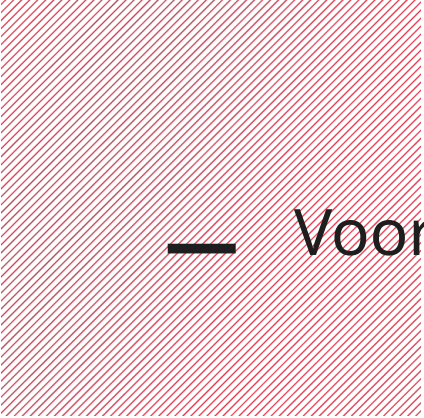
- In Visual Studio Code:
 - Open systeeminfo.sh
 - Rechtsonder: klik op CRLF, verander naar LF
 - Sla opnieuw op
- Herbouw de image
 - Docker build -t systeeminfo:2 .
- Run opnieuw
 - Docker run systeeminfo:2

— WORKDIR in Docker

De instructie WORKDIR stelt de werkdirectory **binnen de container** in.
Vanaf dat punt:

- worden alle relatieve paden geïnterpreteerd t.o.v. deze map
- worden commando's zoals RUN, COPY, CMD uitgevoerd in deze map

Als de map niet bestaat, wordt ze automatisch aangemaakt door Docker.



— Voorbeeld WORKDIR

WORKDIR /app
COPY script.sh .

Hier wordt:

- /app aangemaakt in de container
- script.sh gekopieerd naar /app, niet naar de root

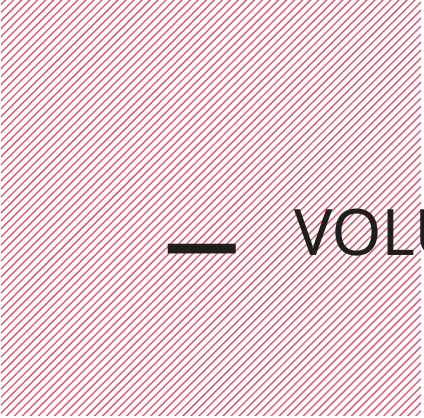




— WORKDIR – opmerkingen

- WORKDIR verwijst alleen naar een map binnen de container
- Het heeft niets te maken met een map op de hostmachine
- Je kunt WORKDIR meerdere keren gebruiken om naar submappen te gaan

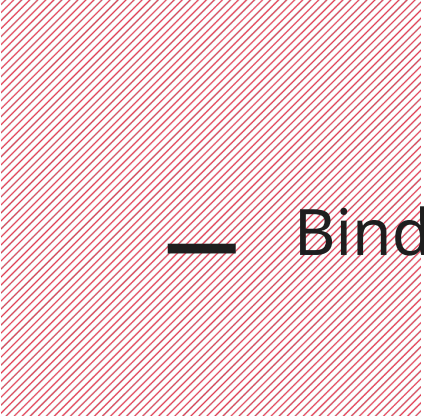




— VOLUME vs BIND MOUNT

Beide maken het mogelijk om **bestanden op de host te koppelen aan een container**, maar er zijn belangrijke verschillen.



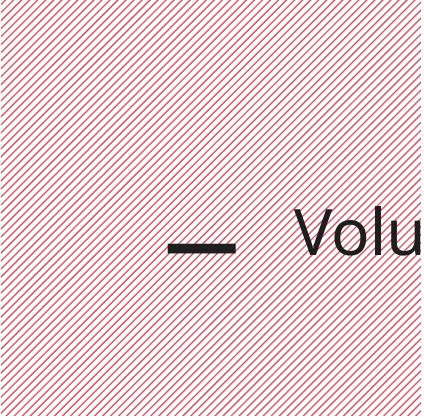


— Bind mount

- Je koppelt een specifiek pad op de host aan een pad in de container
- Volledig zichtbaar en beheerd door de gebruiker
- Goed voor ontwikkelomgevingen

`docker run -v /host/pad:/container/pad image`





— Volume

- Docker beheert zelf de opslaglocatie op de host
- Meestal gebruikt voor persistente data (bv. databases, logs)
- Niet afhankelijk van de mappenstructuur op de host

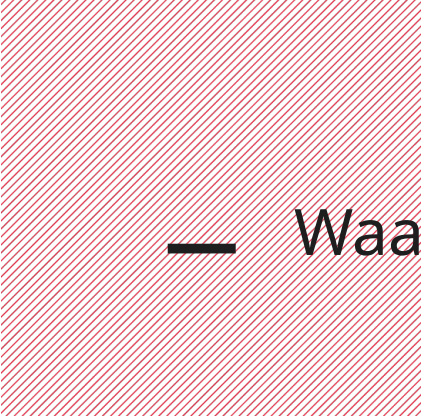
`docker volume create mijnvolume`

`docker run -v mijnvolume:/data image`



— Vergelijking

Kenmerk	Volume	Bind mount
Hostpad kiezen	Nee (Docker kiest pad)	Ja (je kiest pad zelf)
Veiligheid	Meer geïsoleerd	Volledig toegang tot host
Gebruik	Productie, data-opslag	Ontwikkeling, code delen



— Waarom poorten openstellen?

Docker-containers zijn **standaard geïsoleerd** van de buitenwereld.

Als je een webserver of andere service in een container draait, moet je expliciet poorten **publiceren** om die van buitenaf bereikbaar te maken.



— Voorbeeld: webserver in container

Een typische webserver luistert binnen de container op poort 80.

Om die beschikbaar te maken op de host, gebruik je bij docker run:

```
docker run -p 8080:80 nginx
```

Betekenis van -p:

- 8080: poort op de host
- 80: poort in de container

Je kan nu surfen naar <http://localhost:8080> om de webserver te bereiken.

— Meerdere poorten publiceren

Je kunt meerdere poorten publiceren:

```
docker run -p 8080:80 -p 8443:443 nginx
```

- poort 80 → 8080 op de host
- poort 443 (HTTPS) → 8443 op de host

— Wat als je geen -p opgeeft?

Als je geen -p of --publish gebruikt:

- blijft de service enkel bereikbaar binnen de container
- je kunt niet van buitenaf (bv. browser of curl op je pc) verbinden

Gebruik dit commando om te zien welke containers welke poorten gebruiken:

```
docker ps
```

De kolom PORTS toont welke poorten op de host gekoppeld zijn aan poorten in de container.

— Wat zijn multi-container applicaties?

In grotere projecten gebruik je meestal **meerdere containers** die samen één toepassing vormen.

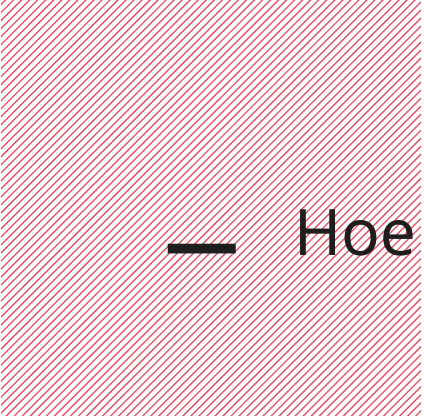
Voorbeeld:

- Een **webserver** container
- Een **databank** container
- Eventueel een **backend API** container

Deze containers moeten met elkaar kunnen communiceren als één geheel.

Voordelen:

- Elke container doet **één taak goed** (single responsibility)
- Je kunt onderdelen **apart updaten, herstarten of schalen**
- Duidelijke **scheiding van componenten**



— Hoe communiceren containers?

Containers kunnen:

- Elkaar bereiken via een Docker-netwerk
- Elkaar aanspreken via containernaam als hostname

Bijvoorbeeld:

de webapp container gebruikt db:5432 om verbinding te maken met de databasecontainer db.

— Hoe beheer je meerdere containers?

Gebruik **Docker Compose** om:

- Meerdere containers samen te definiëren in één bestand (docker-compose.yml)
- Alles met één commando te starten of te stoppen

`docker compose up`

`docker compose down`

— docker-compose.yml – voorbeeldstructuuryaml

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: geheim
```

- web en db zijn twee containers (hostnames)
- Docker regelt automatisch een intern netwerk tussen de services